
Assignment 4

COMP 2402 AB - Fall 2021

Assignment #4

Due: Sunday November 21, 23:59

Submit early and often.

PLEASE REMOVE ALL PRINT STATEMENTS THAT ARE NOT NEEDED.

No late submissions will be accepted.

Academic Integrity

You may:

- Discuss general approaches with course staff,
- Discuss general approaches with your classmates,
- Use code and/or ideas from the textbook,
- Use a search engine / the internet to look up basic Java syntax,
- Send your code / screen share your code with course staff (although it is unlikely the course staff has the bandwidth to look at individuals' code, unfortunately.)

You may not:

- Send or otherwise share code or code snippets with classmates,
- Use code not written by you, unless it is code from the textbook (and you should cite it in comments),
- Use a search engine / the internet to look up approaches to the assignment,
- Use code from previous iterations of the course, unless it was solely written by you,
- Use the internet to find source code or videos that give solutions to the assignment.

If you ever have any questions about what is or is not allowable regarding academic integrity, please do not hesitate to reach out to course staff. We will be happy to answer. Sometimes it is difficult to determine the exact line, but if you cross it the punishment is severe and out of our hands. Any student caught violating academic integrity, whether intentionally or not, will be reported to the Associate Dean and be penalized as follows:

- First offence: F in the course.
- Second offence: One-year suspension from program.
- Third offence: Expulsion from the University.

Assignment 4

These are standard penalties. More-severe penalties will be applied in cases of egregious offences. For more information, please see Carleton University's [Academic Integrity](#) page.

Grading

This assignment will be tested and graded by a computer program (and **you can submit as many times as you like, your highest grade is recorded**). For this to work, there are some important rules you must follow:

- Keep the directory structure of the provided zip file. **If you find a file in the subdirectory comp2402a4 leave it there.**
- Keep the package structure of the provided zip file. **If you find a package comp2402a4; directive at the top of a file, leave it there.**
- Do not rename or change the visibility of any methods already present. If a method or class is public leave it that way.
- Do not change the main(String) method of any class. Some of these are setup to read command line arguments and/or open input and output files. Don't change this behaviour. Instead, learn how to use command-line arguments to do your own testing.
- **Submit early and often.** The submission server compiles and runs your code, and gives you a mark. You can submit as often as you like and only your latest submission will count. There is no excuse for submitting code that does not compile or does not pass tests.
- Write efficient code. The submission server places a limit on how much time it will spend executing your code, even on inputs with a million lines. For some questions it also places a limit on how much memory your code can use. If you choose and use your data structures correctly, your code will easily execute within the time limit. Choose the wrong data structure, or use it the wrong way, and your code will be too slow for the submission server to grade (resulting in a grade of 0).

Submitting and Server Tests

The submission server is available [here](#). If you have any issues, please post on piazza with a clear description of the problem (the text of the error message) and we'll help you as soon as we can. As the deadline approaches, we may have to make changes to the server (see changes described below), but I'm leaving it as-is for now, while its performance holds up.

When you submit your code, the server runs tests on your code similar to how the local tests work. These are bigger and better tests than the small number of tests provided in the "Local Tests" section further down this document. They are obfuscated from you, because you should try to find exhaustive tests of your own code. This can be frustrating because you are still learning to think critically about your own code, to figure out where its weaknesses are. But have patience with yourself and make sure you read the question carefully to understand its edge cases.

Assignment 4

Warning: Do not wait until the last minute to submit your assignment. There is a hard 5 second limit on the time each test has to complete. If the server is heavily loaded, borderline tests may start to fail. **You can submit multiple times and your best score is recorded.**

Changes from Assignment 3's Server:

- There are fewer tests, to hopefully speed things up
- For Part C, all the points are for performance, since correct (but poorly performing) solutions are provided to you.

The Assignment

Purpose & Goals

Generally, the assignments in this course are meant to give you the opportunity to practice with the topics of this course in a way that is challenging yet also manageable. At times you may struggle and at others it may seem more straight-forward; just remember to keep trying and practicing, and over time you will improve.

Specifically, assignment 4 aims to improve your understanding of the tree-based data structures we've been talking about by:

- Encouraging you to think about how you need to use and access data before choosing the interface / data structure to store the data, with a focus on thinking about which of a [List](#), [SSet](#), and [PriorityQueue](#) is more appropriate, in a variety of problems,
- Implementing parts of the BinaryTree interface iteratively (i.e. without recursion),
- Implementing parts of the SSet interface using a Binary Search Tree as the backing data type, using some of the concepts we've been learning about. Hopefully you will contrast these solutions with the array-based and linked-list-based data types to see where they must be different.
- Practicing making changes to an existing code base where you don't understand all of it and you have to ignore what you don't need (learn to just understand "enough" for what you need to do")

Part A: The Setup

Start by downloading and decompressing the [Assignment 4 Zip File \(comp2402a4.zip\)](#), which contains a of the code you need to write. You should have the files: Part1.java, Part2.java, Part3.java, Part4.java, BinaryTree.java, BinarySearchTree.java, SSet.java, DefaultComparator skeleton.java, and Tester.java.

Assignment 4

As with assignments 1, 2, and 3, you can test Part B using the command line or input/output files. You can test Part C with the Tester class included in the zip file. **As before, you'll want to add your own tests as this is just given as a basis on which you should build.** The submission server will do thorough testing; it is your job to read the specifications as carefully as possible, and to examine your own code for potential problems and test those cases.

If you are having trouble running these programs, **figure this out first before attempting to do the assignment.** Check the assignment 4 FAQ on piazza and ask a question there if you are stuck and course staff or another student will likely help you fairly quickly.

Part B: The Interface

As with assignments 1, 2 and 3, you will be filling in the `doIt` method in each of the appropriate `PartX` classes. You can go back to assignments 1, 2 and 3 as reference, and you may use the sample solutions as a starting point if you like.

You can download some sample input and output files for each question as a [zip file \(a4-io.zip\)](#). If you find that a particular question is unclear, you can possibly clarify it by looking at the sample files for that question. **Note that these are very limited tests; you can and should add to them to test your code more thoroughly** as the server tests are much more extensive.

For each of these problems, greater than and less than are with respect to the ordering defined by `String.compareTo()`.

1. [5 marks] Read the input one line at a time, and for each line, output the minimum of all lines so far (including this one) that start with 'c'. If no minimum exists, print a '*' on its own line. For example, input lines `berry`, `apple`, `cheer`, `car`, `add` would result in the output `*`, `*`, `cheer`, `car`, `car`.
2. [10 marks] Read the input one line at a time, and for each line, output the minimum of all lines so far that start with 'c'. If you ever read in the line '###' remove the current minimum line that starts with 'c' before printing the (new) minimum, and continue. As with question 1, if no minimum exists, print a '*' on its own line. When removing the minimum line, if no such minimum exists, just continue. For example, input lines `"berry"`, `"apple"`, `"cheer"`, `"car"`, `"car"`, `"###"`, `"###"` `"add"` would result in the output `"*"`, `"*"`, `"cheer"`, `"car"`, `"car"`, `"car"`, `"cheer"`, `"cheer"`.
3. [10 marks] For this question, you may assume that every input line is distinct. A "step" is a line such that every line that follows it is greater than it. Read in the lines one at a time, and output all the steps. You may assume all lines are distinct. For example, input lines `"f"`, `"d"`, `"a"`, `"c"`, `"b"` would output `"a"`, `"b"`.

Assignment 4

4. [5 marks] For this question, you may assume that every input line is distinct. Read the input one line at a time, and output the line that has x lines greater than it, where $x \geq 0$ is a parameter to the `doIt` method. If there are $\leq x$ lines, output nothing. For example, input lines "a", "b", "d", "c" and $x=3$ would output "a", whereas $x=1$ would output "c".

Part C: The Implementation*Files Provided*

The starter code contains the `BinaryTree`, `BinarySearchTree`, `SSet`, and `DefaultComparator` classes, where `BinarySearchTree` extends from `BinaryTree` (because a BST is a `BinaryTree`), and `BinarySearchTree` implements the provided `SSet` interface. You will not need anything from `SSet` or `DefaultComparator` so you can pretend they don't exist. Part C involves you completing implementations of 5 methods within these two classes, most of which will be familiar to you. These are described in more detail below.

It may help to visualize some trees to help you construct your algorithms and debug. Here are some tools: [tool1](#) and [tool2](#).

Your Tasks

1. [10 marks] In the `BinaryTree` class, implement the method `height2()` without recursion. This method should return the height of the tree (which is -1 for the empty tree). The `BinaryTree` class already has a working `height()` method that uses recursion; you can check your results against it (the `Tester` class does this on one example to get you started.) Since a correct method exists, all marks are for getting an $O(n)$ iterative method.
2. [10 marks] Modify the `BinarySearchTree` class such that each node `u` keeps track of the size of the subtree rooted at `u` in the `BSTNode` class variable `u.s`. Do this such that no runtime increases by more than a constant factor, and `size2(Node u)` should return an accurate `u.s` in $O(1)$ time. Note that the `BinaryTree` class already has a working `size(Node u)` method that uses recursion; you can check your results against it (the `Tester` class gets you started on this.) Since a correct method exists, all marks are for getting an iterative method that does not increase the runtimes of `add` and `remove`.

Hint: Consider changing `s` whenever `n` changes.

3. [10 marks] In the `BinarySearchTree` class, implement the 2 methods

```
public T pred(T x)
public T succ(T x)
```

Assignment 4

that return the predecessor and successor values of x , respectively, that is, the maximum element y in the tree such that $y < x$ and minimum value y such that $y > x$ respectively. If no such element exists, return null. The `BinarySearchTree` class already has a working `slowPred` and `slowSucc` that compute these values in $O(n)$ time. Your goal is to implement them in $O(\text{height})$ time (which in a balanced tree is $O(\log n)$). Again, you can compare your implementation with that of the slow versions (and the `Tester` class does a bit of this.) Since a correct method exists, all marks are for getting an $O(\text{height})$ implementation.

For example, suppose $s = \{2, 4, 6, 8\}$. Then `s.pred(3)` returns 2, `s.pred(2)` returns null, `s.pred(5)` returns 4, and `s.succ(3)` returns 4, `s.succ(8)` returns null, `s.succ(5)` returns 6.

Hint: Note that `succ(x)` is the same as `find(x)` when x is not in the set. You might consider looking at how `find(x)` is implemented and try to use similar logic. Look at the methods that exist in both `BinaryTree` and `BinarySearchTree` to see if anything would help you.

4. [15 marks] (Assignment 3.7 Redux) In the `BST` class, implement

```
public BinarySearchTree<Node,T> chop(T x)
```

that chops our sorted set into two parts at element x . The `SSet this` contains elements $< x$, and the returned `SSet` is a `SSet` that contains elements $\geq x$. This should work for all elements regardless of whether they are in `this`.

For example, suppose $s = \{2, 4, 6, 8\}$. Then `s.chop(3)` returns `{4, 6, 8}` and `s` becomes `{2}`. We would get the same result for `s.chop(4)`.

You can see the `slowChop` method. Note that it is computed correctly, in $O(n)$ time. Your goal is to compute `chop` in $O(h)$ time. (If tree is balanced this is $O(\log n)$ time.)

Local Tests

For Part B, you can download some sample input and output files for each question as a zip file (a4-io.zip). Once you get a sense for how to test your code with these input files, write your own tests that test your program more thoroughly (**the provided tests are not exhaustive!**)

For Part C (`BinaryTree`, `BinarySearchTree`), a `Tester` class is provided with some tests of the provided methods. I recommend you add tests to `Tester.java` to test your `BinaryTree` and `BinarySearchTree` locally (as opposed to on the server).

Assignment 4

Testing suggestions:

- Test incrementally, that is, bit-by-bit,
- Test edge cases, e.g. empty files, empty lists, parameters of odd or even sizes,
- Use small test cases at first, then go bigger only when necessary,
- Test individual methods as you write them, even. As in: you can test an incomplete method to make sure it's doing what you're expected thus far.

Tips, Tricks, and FAQs

This section may be updated as the assignment progresses, but for the most up-to-date FAQ, please see the Assignment 4 FAQ post on discord.

How should I approach each problem?

- Make sure you understand it. Construct small examples, and compute (by hand) the expected output. If you aren't sure what the output should be, go no further until you get clarification.
- Now that you understand what you are supposed to output, and you've been able to solve it by hand, think about how you solved it and whether you could explain it to someone. How about explaining it to a computer?
- If it still seems challenging, what about a simpler case? Can you solve a similar or simplified problem? Maybe a special case? If you were allowed to make certain assumptions, could you do it then? Try constructing your code incrementally, solving the smaller or simpler problems, then, only expanding scope once you're sure your simplified problems are solved.

How should I test my code?

- You should be testing your code as you go along.
- Start small so that you can compute the correct solution by hand.
- Edge cases.
- Large inputs. Random inputs. For time.